AI GTM COMMAND CENTER

Complete Application Build Guide

From scratch to a secure multi-client AI GTM platform, built phase by phase with Claude and free browser-based development tools.

Edition: November 2026 | Audience: solo founder / beginner developer | Approach: MVP first, security before real customer data

PROJECT	TARGET
Product	Multi-tenant GTM agency operating system
Frontend	Next.js, React, Tailwind CSS
Backend	Next.js server routes + Supabase Postgres/Auth
AI	Gemini free-tier API when eligible; Claude for coding
Development	GitHub + browser-based Codespaces
Automation	Server-side rules first; scheduled jobs later
Budget	₹0 prototype within free allowances; production costs not guaranteed

How to use this handbook

- Follow phases in order. Do not ask Claude to build the entire production app in one response.
- At every phase: request code, apply changes, run checks, share errors, and commit only after verification.
- Use synthetic leads and test clients until authentication, RLS, logging and security tests pass.
- Claude is your coding assistant, not automatically your hosting, database, or deployment service.
- This guide is a build specification and prompt workbook, not a claim that the application has already been built.

01 | Scope and product requirements

Build a commercial-grade foundation in small increments. The initial version is an agency internal tool; the later version supports customer-facing accounts and controlled multi-tenancy.

MODULE	MVP	LATER
Dashboard	KPIs, tasks, pipeline totals	Attribution, cohorts, forecasting
Organizations / clients	Agency accounts, membership roles	Client portal, white-label
CRM	Companies, contacts, leads, notes	Deduplication, enrichment
Pipeline	Deals, stages, owners	Forecasting, SLA alerts
Campaigns	Draft campaigns, manual approvals	Provider integrations, experiments
Automation	Lead-created -> task, audit trail	Rule builder, retries, queues
AI assistant	Summaries and draft messages	Agent workflows with tool permissions
Reporting	Basic filters, CSV export	Cross-channel attribution
Settings / security	Auth, RLS, audit, secrets	SSO, advanced controls

User roles

- Platform owner: manages agency-wide configuration and billing, with audited elevated actions.
- Organization admin: manages members, clients and workflows inside their organization.
- Sales operator: works on assigned leads, deals and tasks.
- Viewer: read-only access to permitted organization data.
- Client portal user (later): access only explicitly shared client records.

Non-functional requirements

- No cross-tenant data leakage; enforce access in the database, not only in UI.
- Idempotent workflow triggers; clear retry behavior; traceable logs.
- Mobile-responsive interface, keyboard-accessible forms, input validation, and error states.
- Data export, backup plan, consent/opt-out handling, and documented deletion procedures.

02 | Free tools, accounts and limits

TOOL	PURPOSE / CAUTION
GitHub	Private repository, version control; never commit credentials
GitHub Codespaces	Browser editor and terminal; limited free compute/storage; stop when idle
Next.js + Tailwind	Free open-source app framework and styling
Supabase Free	Auth + Postgres + RLS; free project limits and pausing apply
Claude	Coding guidance; free access may be rate-limited, features vary
Google AI Studio / Gemini API	Optional runtime AI; eligibility, quotas and data terms must be checked
GitHub Actions	CI and limited schedules; not reliable real-time task infrastructure
Local Codespaces preview	Private development preview, not production uptime

Important: Do not rely on Vercel Hobby for a commercial production agency application: verify the hosting provider’s current commercial-use terms. No provider guarantees unlimited free production operations.

03 | Project setup in the browser

- Create a GitHub account at github.com and choose the free personal plan.
- Create a PRIVATE repository: ai-gtm-command-center. Add README.
- Open Code > Codespaces > Create codespace on main. Use the smallest machine.
- Open Terminal in browser-based VS Code and run the commands below.

```
npx create-next-app@latest platform --js --tailwind --eslint --app --src-dir --use-npm --yes
cd platform
npm run dev
```

Use the Ports tab to open port 3000. If port forwarding is exposed publicly, change its visibility to private. Stop the server with Ctrl+C. Save work using git.

```
cd /workspaces/ai-gtm-command-center
git add .
git commit -m "Initialize Next.js app"
git push origin main
```

If Git requests an identity, configure your own GitHub username/email. If the application is created at repository root instead of platform/, adjust all file paths accordingly.

04 | Recommended repository structure

```
platform/
  src/app/               # routes and server endpoints
  src/app/(app)/         # authenticated screens
  src/app/api/           # validated API routes
  src/components/        # UI components
  src/lib/               # auth, database, AI, workflow helpers
  src/types/             # shared types / schemas
  supabase/migrations/   # versioned SQL migrations
  tests/                 # automated tests
  .env.example           # names only; no secret values
  README.md              # setup, run, test, deploy
```

Prefer TypeScript for long-term maintainability, even if the initial starter was created with JavaScript. Claude can migrate incrementally once the base application works.

Phase 1 | Product shell and design system

Outcome: A responsive admin dashboard with real navigation and realistic synthetic data.

- Create layout, sidebar, top bar, dashboard cards, table components, loading/empty/error states.
- Create pages: /dashboard, /clients, /leads, /deals, /campaigns, /automations, /ai, /reports, /settings.
- Use server-safe components; mark only interactive components with "use client".
- Create a reusable color palette and component library; keep pages readable on mobile.

Claude prompt - copy into Claude

Act as a senior Next.js engineer. In my existing Next.js App Router project, implement Phase 1 only: a responsive AI GTM Command Center UI with the listed routes, shared sidebar/topbar, synthetic data, accessible forms and consistent styling. Give exact file paths and full file contents. Do not add a real database yet. Include terminal commands, verification and a Git commit message.

Acceptance test / exit gate

- npm run dev works; all menu links route to pages.
- npm run lint passes; no console errors.
- Screens render on phone and desktop widths.

Phase 2 | Supabase database and authentication

Outcome: Secure sign-in, organizations and persistent records.

- Create Supabase project and copy Project URL and publishable/anon key into local environment secrets.
- Design versioned SQL migrations and membership tables. Enable RLS on every tenant-owned table.
- Implement Supabase SSR auth/session handling; protect authenticated routes server-side.
- Create onboarding flow: sign up, create organization, add owner membership.
- Keep service-role keys server-only; never expose them through NEXT_PUBLIC variables.

Claude prompt - copy into Claude

Implement Phase 2 in my existing Next.js project. Add Supabase Auth, server-side session handling, organizations and organization_members with role-based access. Write versioned SQL migrations and RLS policies using auth.uid() and membership checks. Show exact code and a two-user cross-tenant isolation test. Never expose service role keys. Do not proceed to CRM until the tests pass.

Acceptance test / exit gate

- Unauthenticated user cannot open protected routes.
- User A cannot read or modify User B organization data, even through direct API calls.
- Reload retains authorized data; logout revokes access.

Phase 3 | Client management and CRM

Outcome: Create, edit, search and archive agency clients and sales leads.

- Implement clients, companies, contacts, leads, notes, activities and ownership fields.

- Use Zod or equivalent validation on all writes and safe server-side authorization.
- Add pagination, filtering, search, soft delete and created/updated timestamps.
- Record who created or changed each business record.

Claude prompt - copy into Claude

Implement Phase 3 only: persistent clients, companies, contacts, leads, notes and activities in Supabase. Add migrations, RLS, server actions/API handlers, forms, search, pagination and tests. Use organization_id on all tenant records and validate permissions on every write. Provide exact file changes and acceptance tests.

Acceptance test / exit gate

- Client and lead CRUD persist after refresh.
- Cross-tenant tests remain green.
- Invalid inputs return safe, understandable errors.

Phase 4 | Deals, pipeline and tasks

Outcome: Sales teams can manage opportunities and follow-up actions.

- Create pipelines, stages, deals, task assignments, due dates and activity history.
- Implement Kanban or stage list; validate allowed transitions.
- Compute totals on the server and filter by authorized organization.
- Create overdue and upcoming task views.

Claude prompt - copy into Claude

Implement Phase 4 only: deals, configurable pipeline stages, tasks, owners and audit events. Provide SQL migrations, secure server operations, responsive UI and tests for totals, stage transitions and organization isolation.

Acceptance test / exit gate

- Move a deal and see correct stage and totals.
- Task ownership and overdue filtering work.
- Stage changes create an audit activity.

Phase 5 | Automation engine

Outcome: A reliable first workflow: new lead -> follow-up task + activity.

- Define workflow rules, events, runs, statuses, errors and idempotency keys.
- Trigger automation after a successful lead creation; avoid duplicate task creation.
- Record each run and failure; add safe manual retry controls.
- Start with built-in server-side rules before attempting a visual workflow builder.

Claude prompt - copy into Claude

Implement Phase 5: a minimal server-side automation engine. Start with lead.created -> create follow-up task and activity log. Use workflow_runs, idempotency keys, error handling and tests. Do not use browser timers or claim guaranteed scheduled delivery. Provide exact file paths and test commands.

Acceptance test / exit gate

- Creating one lead produces exactly one follow-up task.
- Repeated event delivery does not duplicate tasks.
- Failed runs show a readable error and retry outcome.

Phase 6 | AI assistant with human review

Outcome: AI can summarize leads and draft outreach, but cannot send autonomously.

- Choose a free-eligible runtime model provider; verify quotas and data-use terms first.
- Keep API key server-side; build an authenticated AI route with rate limits.
- Create prompts for summarization, lead qualification suggestions and email drafts.
- Require human review/approval before any outbound communication.
- Log generation metadata without unnecessary sensitive prompt contents.

Claude prompt - copy into Claude

Implement Phase 6: a provider-isolated AI assistant for lead summaries and email drafts using a free-eligible Gemini API key stored server-side. Add schema validation, usage limits, audit metadata, prompt injection defenses and mandatory human approval before sending. Include a no-key mock mode for free testing.

Acceptance test / exit gate

- A demo lead generates a usable summary and draft.
- No API key is visible in browser or repository.
- Provider errors and exhausted quota display gracefully.

Phase 7 | Campaign management

Outcome: Campaign drafts, target segments and approval controls.

- Create campaigns, segments, campaign_leads and message_drafts.
- Add statuses: draft, awaiting approval, approved, paused, completed.

- Implement opt-out/consent fields and suppression checks.
- Use manual export or approved integrations only; no unsolicited mass-email system in MVP.

Claude prompt - copy into Claude

Implement Phase 7: campaign drafts, segmentation, message templates, approval workflow, suppression/opt-out handling and audit logs. Keep sending disabled by default; build only lawful, consent-aware integrations with explicit user approval.

Acceptance test / exit gate

- Draft campaigns can be created and reviewed.
- Suppressed contacts cannot be queued.
- Every approval is attributed to a user.

Phase 8 | Reports and exports

Outcome: Trustworthy per-client reporting and CSV exports.

- Build metrics: leads by stage, conversion rate, deals by stage, tasks due, automation failures.
- Provide date and client filters and clear metric definitions.
- Add CSV export only for authorized users and authorized organization records.
- Avoid claiming revenue attribution without sufficient source data.

Claude prompt - copy into Claude

Implement Phase 8: authorized reporting queries, filters, dashboard charts and CSV export. Include metric definitions, efficient SQL indexes, reconciliation tests and tenant-isolation checks.

Acceptance test / exit gate

- Report totals reconcile with underlying records.
- Filters and exports cannot leak another organization data.
- Empty datasets display zero/empty states clearly.

Phase 9 | Client portal, operations and deployment

Outcome: Controlled customer access and production readiness.

- Add client-scoped portal roles only after organization isolation is proven.
- Set up CI: lint, typecheck, unit/integration tests, dependency scanning.
- Add monitoring, backups, incident process, retention and deletion policies.
- Review provider commercial-use terms; choose sustainable production hosting.
- Perform manual security review before entering real customer data.

Claude prompt - copy into Claude

Implement Phase 9 as a production-readiness pass. Add client portal permissions, CI, backup/restore runbook, audit logging, rate limiting, security headers, error monitoring, retention controls and deployment checklist. Do not deploy until authorization and RLS tests pass.

Acceptance test / exit gate

- All automated checks pass; cross-tenant penetration tests pass.
- Document restore procedure and secret rotation.
- Real deployment has HTTPS, controlled access and incident owner.

05 | Suggested relational database schema

TABLE	ESSENTIAL FIELDS
organizations	id, name, created_at
organization_members	organization_id, user_id, role, created_at
clients	id, organization_id, name, status, created_at
companies	id, organization_id, client_id, name, website
contacts	id, organization_id, company_id, name, email, opt_out
leads	id, organization_id, client_id, contact_id, status, owner_id
pipelines / stages	id, organization_id, name, sort_order
deals	id, organization_id, lead_id, stage_id, amount, currency
tasks	id, organization_id, lead_id, assignee_id, due_at, status
activities	id, organization_id, actor_id, entity_type, entity_id, action
workflows	id, organization_id, trigger_type, enabled, config
workflow_runs	id, organization_id, workflow_id, event_key, status, error
campaigns	id, organization_id, client_id, status, approved_by
ai_generations	id, organization_id, lead_id, kind, status, created_at

Database design rules

- Use UUID primary keys, foreign keys, indexes on organization_id and frequently filtered fields.
- Use server-generated timestamps and immutable activity/audit records.
- For client-scoped access, enforce BOTH organization membership and permitted client relationship.
- Do not assume that checking organization_id in UI is security. Use RLS and server authorization.
- Use SQL migrations in source control. Never rely only on manual dashboard edits.

06 | Claude master prompt

Paste this once at the start of a fresh Claude project or conversation, then use the individual phase prompts above:

MASTER PROMPT: You are my senior full-stack architect and patient coding instructor. We are building AI GTM Command Center, a secure multi-tenant B2B GTM agency operating system. Stack: Next.js App Router, Tailwind, Supabase Auth/Postgres/RLS, server-side automation, optional Gemini AI. I am using GitHub Codespaces in a browser and want to stay within free-tier allowances. Work one phase at a time. Before coding, inspect the files I paste and state assumptions. For every change, provide exact file paths, full

replacement code or precise patches, shell commands, environment variable names without secret values, database migrations, security considerations, tests, and a commit message. Never claim you executed commands you cannot execute. Do not proceed until I confirm acceptance tests. Prioritize tenant isolation, safe defaults and mobile-friendly UI.

How to share project context with Claude

- Paste the current folder tree, package.json, and only the relevant source files.
- Paste terminal errors as text; remove API keys, passwords, personal data and session tokens.
- Ask Claude to modify one module at a time, not regenerate the whole project.
- Ask for tests first when debugging security or workflow logic.

07 | Practical terminal commands

```
cd /workspaces/ai-gtm-command-center/platform
npm install
npm run dev
npm run lint
npm run build
git status
```

The project may not have a test script until Claude adds one. Never assume npm test is configured. Use the scripts listed in package.json.

08 | Environment variables

```
# platform/.env.local (NEVER COMMIT)
NEXT_PUBLIC_SUPABASE_URL=your_project_url
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_publishable_key
GEMINI_API_KEY=server_side_key_only
```

Use the current Supabase key naming convention shown in your project dashboard; the legacy anon key and newer publishable key are not always labeled identically. Never expose privileged service-role credentials.

09 | QA and security checklist

- RLS enabled on all organization-owned tables; two-user isolation tests passed.
- No private credentials in git history or browser bundles.
- All write APIs validate inputs, auth and organization membership.
- Workflows are idempotent; retries and failures are logged.
- AI outputs are reviewed by humans; no autonomous unsolicited outreach.
- Suppression/opt-out logic and appropriate data handling are in place.
- Mobile UI, accessibility, error states, backups and restore tests are documented.
- Hosting permits intended commercial use; quota alerts and costs are understood.

10 | Timeline and realistic budget

MILESTONE	PLANNING WINDOW	COST NOTE
UI + local prototype	Week 1-2	Free within Codespaces quota
Auth + CRM + pipeline	Week 3-5	Supabase free tier if eligible
Automation + AI drafts	Week 6-8	AI free quota if eligible
Campaigns + reports	Week 9-10	Provider integrations may cost extra
Security + pilot readiness	Week 11-12+	Commercial hosting not guaranteed free

These are indicative timelines, not delivery guarantees. A beginner working part time may take longer. Free tiers can change; monitor account billing settings and disable paid overages.

11 | Next action - today

- Create a private GitHub repository and Codespace.
- Create the Next.js app in platform/ and open the preview.
- Paste the Phase 1 Claude prompt and request file-by-file code.
- Run lint/build and verify navigation; commit changes.
- Only then create Supabase and move to Phase 2.

Reference links

https://github.com/codespaces
https://nextjs.org/docs
https://supabase.com/docs
https://docs.github.com/en/codespaces
https://docs.anthropic.com/
https://ai.google.dev/gemini-api/docs
https://docs.github.com/en/actions

Note: Check the latest product documentation, quotas and terms before adopting any provider. This document is a practical architecture and implementation plan, not a vendor endorsement.